

一维前缀和 & 差分算法 | 例子讲解课件

适用年级： GESP 三级 ~ 四级

课时安排： 1 ~ 2 课时（建议连堂或分两次讲）

目录

- [Part 1: 引子——从一道天真的题开始](#)
- [Part 2: 前缀和 —— 预先把"和"算好](#)
- [Part 3: 课堂练习 ①](#)
- [Part 4: 差分 —— 反过来想问题](#)
- [Part 5: 课堂练习 ②](#)
- [Part 6: 前缀和 vs 差分 —— 双胞胎对比](#)
- [Part 7: 两道实战例题](#)
- [Part 8: 本课小结 & 易错点](#)

Part 1: 引子——从一道天真的题开始

问题

有一个数列 a ，共 n 个数。现在有 m 次询问，每次问你 $a[l]$ 到 $a[r]$ 的和，怎么做？

天真的做法

```
1 | for (int i = l; i <= r; i++) sum += a[i];
```

每次询问都从 l 循环到 r ，把所有数加一遍。

时间： $O(n \times m)$

- $n = 10^5, m = 10^5 \rightarrow$ 最坏 10^{10} 次运算 $\rightarrow$ **完蛋**。

能不能快一点？

仔细想想——每次都要把 l 到 r 的每个数当场加一遍，但中间大量数字被重复加了。

核心思想： 能不能提前把所有区间和都算好存下来？

比如：

- 存好 $a[1] \sim a[1]$ 的和
- 存好 $a[1] \sim a[2]$ 的和
- 存好 $a[1] \sim a[3]$ 的和
-
- 存好 $a[1] \sim a[n]$ 的和

那要算 $a[l] \sim a[r]$ 的和时，是不是有办法直接"减"出来？

👉 这就是前缀和的思想。

Part 2：前缀和 —— 预先把"和"算好

定义

定义一个新数组 `pre`，其中：

```
1 | pre[0] = 0
2 | pre[i] = a[1] + a[2] + ... + a[i]    (i = 1..n)
```

用递推写就是：

```
1 | pre[i] = pre[i-1] + a[i]
```

构造过程举例

1	a:	[_, 3, 1, 4, 1, 5, 9, 2, 6]	(下标从 1 开始)
2		↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑	
3		1 2 3 4 5 6 7 8	

i	a[i]	pre[i] = pre[i-1] + a[i]
0	—	0
1	3	0 + 3 = 3
2	1	3 + 1 = 4
3	4	4 + 4 = 8
4	1	8 + 1 = 9
5	5	9 + 5 = 14
6	9	14 + 9 = 23
7	2	23 + 2 = 25
8	6	25 + 6 = 31

1	pre:	[0, 3, 4, 8, 9, 14, 23, 25, 31]
---	------	---------------------------------

核心公式：区间和查询

1	sum[l..r] = pre[r] - pre[l-1]
---	-------------------------------

图解验证：想求 `a[3] + a[4] + a[5]` (即下标 3 到 5 的和)

```

1 pre[5] = a[1]+a[2]+a[3]+a[4]+a[5] = 14
2 pre[2] = a[1]+a[2] = 4
3 -----
4 相减      a[3]+a[4]+a[5] = 10 ← 正好

```

验证数值: $a[3]=4 + a[4]=1 + a[5]=5 = 10 \checkmark$

为什么 pre[0] 要设为 0?

为了公式统一。如果 $l = 1$, $pre[1] - pre[0] = pre[1] - 0 = a[1]$, 公式依然成立。没有 $pre[0] = 0$, $l = 1$ 就要特判。

代码模板

```

1  #include <iostream>
2  using namespace std;
3
4  const int N = 100010;
5  int a[N], pre[N];
6
7  int main() {
8      int n, m;
9      cin >> n >> m;
10     for (int i = 1; i <= n; i++) {
11         cin >> a[i];
12         pre[i] = pre[i-1] + a[i];           // 边
13     }
14     while (m--) {
15         int l, r;
16         cin >> l >> r;
17         cout << pre[r] - pre[l-1] << endl; //
18     }
19     return 0;
20 }

```

时间复杂度： $O(n + m)$ ，比天真的 $O(n \times m)$ 快了一个数量级。

Part 3：课堂练习 ①

练习 1.1 —— 热身

数组 $a = [2, 7, 1, 3, 8, 4]$ ，计算前缀和数组 pre 。

i	1	2	3	4	5	6
$a[i]$	2	7	1	3	8	4
$pre[i]$	?	?	?	?	?	?

然后回答： $a[2] \sim a[5]$ 的和是多少？

► 点击看答案

$pre = [0, 2, 9, 10, 13, 21, 25]$

$a[2] \sim a[5] = pre[5] - pre[1] = 21 - 2 = 19$

验证： $7 + 1 + 3 + 8 = 19 \checkmark$

练习 1.2 —— 动手编

输入一个 n 个数的数组，然后 m 次询问，每次输出区间和。

```
1 输入：
2 5 3
3 1 2 3 4 5
4 1 3
5 2 5
6 3 4
7
8 输出：
9 6
10 14
11 7
```

► 点击看答案

`pre = [0, 1, 3, 6, 10, 15]`

1~3: $\text{pre}[3] - \text{pre}[0] = 6 - 0 = 6$

2~5: $\text{pre}[5] - \text{pre}[1] = 15 - 1 = 14$

3~4: $\text{pre}[4] - \text{pre}[2] = 10 - 3 = 7$

Part 4: 差分 —— 反过来想问题

新问题

有一个数组 `a`，初始全部为 0。现在要做 `m` 次操作，每次操作把 `a[l]` 到 `a[r]` 的所有数加上同一个值 `x`。所有操作结束后，输出最终的数组。

天真的做法

```
1 for (int i = l; i <= r; i++) a[i] += x;
```

每次 $O(r-l+1)$ ， m 次最坏 $O(n \times m)$ ，又完蛋。

关键观察

如果有一对 **逆运算**，其中一个"麻烦操作"在另一个的世界里就变成了"简单操作"——

前缀和的逆运算是什么？

让我们理一理。

已知 pre 是 a 的前缀和：

```
1 | pre[i] = a[1] + a[2] + ... + a[i]
```

那反过来说：

```
1 | a[1] = pre[1]
2 | a[2] = pre[2] - pre[1]
3 | a[3] = pre[3] - pre[2]
4 | ...
5 | a[i] = pre[i] - pre[i-1]
```

这个逆运算就叫 **差分**。

差分的定义

定义差分数组 `diff`：

```
1 | diff[1] = a[1]
2 | diff[i] = a[i] - a[i-1]    (i >= 2)
```

验证：对 $diff$ 求前缀和，能得到原数组 a 吗？

```
1 | preDiff[1] = diff[1] = a[1]
2 | preDiff[2] = diff[1] + diff[2] = a[1] + (a[2] - a[1]) = a[2]
3 | preDiff[3] = diff[1] + diff[2] + diff[3] = a[1] + (a[2] - a[1]) + (a[3] - a[2]) = a[3]
```

✓ 把 diff 求一遍前缀和，就还原回了原数组 a。

核心魔法：区间加如何变成 $O(1)$

回到问题：想给 $a[l] \sim a[r]$ 每个数加 x 。

操作：

```
1 diff[l] += x;
2 diff[r+1] -= x;
```

为什么？

思想方向：计算过程是 **diff** → **求前缀和** → **得到 a**。

- 在 $\text{diff}[l]$ 处加 $x \rightarrow$ 前缀和一路传到后面，所有 $\geq l$ 的 $a[i]$ 都多了 x
- 但我不想影响 r 之后的数，所以在 $\text{diff}[r+1]$ 处减 $x \rightarrow$ 到 $r+1$ 开始又抵消了

效果：

1	位置：	1	1+1	...	r	r+1	r+2
2	-----						
3	diff:	+x	0	...	0	-x	0
4	前缀和：	+x	+x	...	+x	0	0
5						↑	
6						刚好到 r 为止 +x	

神奇吗？其实是数学在背后撑腰。

差分构造举例

$a = [0, 0, 0, 0, 0, 0]$ ，三次操作：

1. $[2, 4]$ 加 3

2. [1, 3] 加 2

3. [3, 5] 加 5

操作 diff 数组:

初始 diff = [0, 0, 0, 0, 0, 0, 0] (要开到 $n+2$, 因为可能操作到 $r+1$)

- 1 操作1: [2,4] +3 → diff[2]+=3, diff[5]-=3
- 2 操作2: [1,3] +2 → diff[1]+=2, diff[4]-=2
- 3 操作3: [3,5] +5 → diff[3]+=5, diff[6]-=5

位置	1	2	3	4	5	6
初始 diff	0	0	0	0	0	0
操作1后	0	+3	0	0	-3	0
操作2后	+2	+3	0	-2	-3	0
操作3后	+2	+3	+5	-2	-3	-5

求前缀和还原 a:

- 1 $a[1] = 0 + 2 = 2$
- 2 $a[2] = 2 + 3 = 5$
- 3 $a[3] = 5 + 5 = 10$
- 4 $a[4] = 10 + (-2) = 8$
- 5 $a[5] = 8 + (-3) = 5$
- 6 $a[6] = 5 + (-5) = 0$

结果: $a = [2, 5, 10, 8, 5, 0]$

手动验证:

- 原数组全 0
- 操作1 [2,4]+3 → [0,3,3,3,0,0]
- 操作2 [1,3]+2 → [2,5,5,3,0,0]

- 操作3 $[3,5]+5 \rightarrow [2,5,10,8,5,0]$ ✓

代码模板

```
1  #include <iostream>
2  using namespace std;
3
4  const int N = 100010;
5  int diff[N];    // 直接用 diff 数组（初始全 0）
6
7  int main() {
8      int n, m;
9      cin >> n >> m;
10     // 如果初始数组不是全 0，先通过 diff 构造出来
11     // for (int i = 1; i <= n; i++) {
12     //     int x; cin >> x;
13     //     diff[i] += x;           // 在 i 处加 x，
    然后在 i+1 处减 x → 等价于放一个数
14     //     diff[i+1] -= x;
15     // }
16     while (m--) {
17         int l, r, x;
18         cin >> l >> r >> x;
19         diff[l] += x;
20         diff[r+1] -= x;
21     }
22     // 最后求前缀和得到最终数组
23     for (int i = 1; i <= n; i++) {
24         diff[i] += diff[i-1];    // 原地前缀和
25         cout << diff[i] << " \n"[i==n];
26     }
27     return 0;
28 }
```

时间复杂度： $O(n + m)$ ，每次区间加 $O(1)$ 。

Part 5: 课堂练习 ②

练习 2.1 —— 小试

数组 a 初始全 0，进行以下操作后输出最终数组：

- [1, 6] 加 1
- [2, 5] 加 2
- [3, 4] 加 3

► 点击看答案

diff 处理：

操作	diff
[1,6]+1	diff[1]=1, diff[7]=-1
[2,5]+2	diff[2]=2, diff[6]=-2
[3,4]+3	diff[3]=3, diff[5]=-3

累加：diff = [1, 2, 3, 0, -3, -2, -1]

前缀和还原：

a[1]=1, a[2]=3, a[3]=6, a[4]=6, a[5]=3, a[6]=1

结果：a = [1, 3, 6, 6, 3, 1]

验证：手动算加法

- 操作1: [1,1,1,1,1,1]
- 操作2: +[0,2,2,2,2,0] → [1,3,3,3,3,1]
- 操作3: +[0,0,3,3,0,0] → [1,3,6,6,3,1] ✓

练习 2.2 —— 数组初始有值怎么办？

`a = [1, 3, 5, 7, 9]`

然后操作 `[2, 4]` 加 10

最终数组是什么？

提示： 可以先把 `a` 通过差分"放"进 `diff`，再做区间加，最后求前缀和还原。

► 点击看答案

先把原始数组放入 `diff`：

```
1 放 a[1]=1: diff[1]+=1, diff[2]-=1 → diff=[1,-1,0,0,0,0]
2 放 a[2]=3: diff[2]+=3, diff[3]-=3 → diff=[1,2,-3,0,0,0]
3 放 a[3]=5: diff[3]+=5, diff[4]-=5 → diff=[1,2,2,-5,0,0]
4 放 a[4]=7: diff[4]+=7, diff[5]-=7 → diff=[1,2,2,2,-7,0]
5 放 a[5]=9: diff[5]+=9, diff[6]-=9 → diff=[1,2,2,2,2,-9]
```

操作 `[2,4]+10`：`diff[2]+=10, diff[5]-=10`

最终 `diff = [1, 12, 2, 2, -8, -9]`

前缀和还原：

`a[1]=1, a[2]=13, a[3]=15, a[4]=17, a[5]=9`

验证：原 `[1,3,5,7,9]` $\rightarrow$ `[2,4]+10` $\rightarrow$ `[1,13,15,17,9]` ✓

Part 6：前缀和 vs 差分 —— 双胞胎对比

特性	前缀和 (Prefix Sum)	差分 (Difference)
是什么	把和提前算好存着	记录相邻元素的差值
公式	$pre[i] = pre[i-1] + a[i]$	$diff[i] = a[i] - a[i-1]$
关系	—	对 diff 求前缀和 = 原数组 a
核心操作	$O(1)$ 求 $a[l] \sim a[r]$ 的和	$O(1)$ 给 $a[l] \sim a[r]$ 加一个数
操作公式	$sum = pre[r] - pre[l-1]$	$diff[l] += x, diff[r+1] -= x$
适合场景	多次区间查和	多次区间加值
为什么快	预计算，空间换时间	把区间操作转化为端点操作

一句话理解

- **前缀和**：先把历史算好存着，每次查的时候直接减一下（预处理 → 快速查询）
- **差分**：先把要改的地方标记在端点，最后一起算（标记 → 统一还原）

它俩是逆运算，知道一个就能推出另一个。

Part 7：两道实战例题

例 1 —— 最高海拔（前缀和应用）

一位骑手从海拔 0 开始骑行。已知 n 段路程中每段的海拔变化量 $gain[i]$ （可为正或负），求骑行过程中达到的**最高海拔**。

输入: gain = [-5, 1, 5, 0, -7]
输出: 1

思路: 海拔数组是 gain 的前缀和。

```
1 海拔: 0 → (-5) → (-4) → 1 → 1 → (-6)
2                ↑
3                最高 1
```

直接求前缀和, 同时记录最大值。

```
1 int largestAltitude(vector<int>& gain) {
2     int cur = 0, ans = 0;
3     for (int x : gain) {
4         cur += x;           // 递推前缀和 (当前位置的海拔)
5         ans = max(ans, cur);
6     }
7     return ans;
8 }
```

例 2 —— 航班预订统计 (差分应用)

有 n 个航班, 编号 $1 \sim n$ 。给你一个 bookings 数组, 每个 bookings[i] = [l, r, x] 表示预订了从 l 到 r 的每个航班 x 个座位。

返回长度为 n 的数组 answer, answer[i] 表示航班 i 的总预订数。

这就是典型的 **多次区间加** → **最后输出** 场景, 差分完全适用。

```
1 vector<int>
  corpFlightBookings(vector<vector<int>>& bookings,
    int n) {
```

```

2    vector<int> diff(n + 2, 0); // 多开一些，避免
    越界
3    for (auto& b : bookings) {
4        int l = b[0], r = b[1], x = b[2];
5        diff[l] += x;
6        diff[r+1] -= x;
7    }
8    vector<int> ans(n);
9    int cur = 0;
10   for (int i = 1; i <= n; i++) {
11       cur += diff[i]; // 前缀和还原
12       ans[i-1] = cur;
13   }
14   return ans;
15 }

```

Part 8: 本课小结 & 易错点

✓ 必须记住的

公式	用途
<code>pre[i] = pre[i-1] + a[i]</code>	构造前缀和
<code>sum[l..r] = pre[r] - pre[l-1]</code>	查询区间和
<code>diff[i] = a[i] - a[i-1]</code>	构造差分
<code>diff[l] += x, diff[r+1] -= x</code>	区间加操作
对 diff 求前缀和 = 原数组	差分还原

● 常见易错

错误	正确	原因
数组开到 n 刚好	开到 $n+2$	<code>diff[r+1]</code> 会访问 $n+1$
忘记 long long	结果或 pre 用 long long	$n=10^5$ 、值= 10^9 时 <code>pre[n]</code> 可能爆 int
<code>pre[i]</code> 写成 <code>pre[i] +=</code> <code>a[i]</code>	<code>pre[i] =</code> <code>pre[i-1] + a[i]</code>	不是累加，是递推
差分时 <code>diff[r]</code> <code>-= x</code>	<code>diff[r+1] -= x</code>	是 r 的下一个位置才开始抵消
多组测试不重置 <code>pre[0]</code>	<code>pre[0] = 0</code> 每次清空	上一组的数据会干扰下一组

💡 选讲（时间够就提一下）

- 二维前缀和 / 差分是同一个思想的二维扩展，后续再讲
- 前缀和还可以做 **子数组和等于 k** 的问题（配合哈希表）
- 差分还能做 **扫描线、区间覆盖** 等问题

📖 课件编号：7_28_1

📅 日期：2026 年 7 月 28 日

🖋️ 制作者：🏠 丘

附录：课堂板书建议（老师参考）

白板布局建议

1		
2	前缀和	差分
3		
4	$a = [3, 1, 4, 1, 5]$	$diff[1] =$
5	$pre[0] = 0$	$diff[i] =$
6	$pre[i] = pre[i-1] + a[i]$	
7	$pre = [0, 3, 4, 8, 9, 14]$	操作: $[l, r] +$
8		$\rightarrow diff[l] += x$
9	查 $[l, r]$ 和:	
10	$diff[r+1] -= x$	
11		最终: $diff$ 求
12	前缀和 = a	
13	$\leftarrow$ 查和很快	$\leftarrow$ 改值很快
14		
15	核心关系: 差分 $\rightleftharpoons$ 前缀和 (互为逆运算)	
16	差分求一遍前缀和 = 原数组	
	原数组求一遍前缀和 = 前缀和数组	

建议讲解节奏

阶段	时间	内容
引子 + 思考	5min	抛问题，让学生想天河做法 vs 聪明做法
前缀和推导	10min	公式推导 + 板书画格子演示
练习 1	5min	课堂练笔
休息 / 转换	3min	"那如果问题反过来呢?" 引出差分
差分推导	12min	diff定义 → 验证还原 → 核心魔法操作
练习 2	5min	课堂练笔
对比总结	5min	画对比表 + 易错点强调
实战例题	10min	1~2 道 LeetCode 经典题